



SQL Interview Practice Guide

PostgreSQL – Scenario Based Questions with Solutions

Curated by Ashish Zope

📊 Beginner → Intermediate → Advanced → Expert → Critical Thinking
💡 Real-world scenarios • Complete schema • Full solutions

What You'll Master

- ✓ **100 curated questions** from beginner to expert level
- ✓ **JOINS, Aggregations, Subqueries** & Window Functions
- ✓ **CTEs, Transactions, Triggers** & Advanced PostgreSQL
- ✓ **Complete CompanyDB schema** with sample data
- ✓ **Full SQL solutions** tested & production-ready
- ✓ **Real interview scenarios** & best practices

Note: The complete database schema and sample data are provided at the end of this document. Load the **CompanyDB** schema into your PostgreSQL instance before practicing these questions. All queries are tested against the included dataset.

Beginner Level

Foundation questions covering basic SELECT, WHERE, JOIN, and GROUP BY operations

1. List all employees with their department names

```
SELECT e.EmployeeID, e.FirstName, e.LastName, d.DepartmentName
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID;
```

2. Find the highest salary among employees

```
SELECT MAX(Salary) AS HighestSalary FROM Employees;
```

3. Show all projects with budget greater than 200,000

```
SELECT ProjectName, Budget FROM Projects WHERE Budget > 200000;
```

4. Count total employees in the company

```
SELECT COUNT(*) AS TotalEmployees FROM Employees;
```

5. List employees hired after 2020

```
SELECT FirstName, LastName, HireDate FROM Employees WHERE HireDate > '2020-01-01';
```

6. Show all distinct department names

```
SELECT DISTINCT DepartmentName FROM Departments;
```

7. Find employees with salary less than 60,000

```
SELECT FirstName, LastName, Salary FROM Employees WHERE Salary < 60000;
```

8. Order employees by salary descending

```
SELECT FirstName, LastName, Salary FROM Employees ORDER BY Salary DESC;
```

9. Show first 2 employees hired

```
SELECT FirstName, LastName, HireDate FROM Employees ORDER BY HireDate ASC LIMIT 2;
```

10. Find employees in IT department

```
SELECT e.FirstName, e.LastName FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE d.DepartmentName = 'IT';
```

11. Show employees with salary greater than 70,000

```
SELECT FirstName, LastName, Salary
FROM Employees
WHERE Salary > 70000;
```

12. Find employees from Jalgaon or Bhusawal

```
SELECT e.FirstName, e.LastName, d.Location
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE d.Location IN ('Jalgaon', 'Bhusawal');
```

13. List employees whose last name is 'Patil'

```
SELECT FirstName, LastName
FROM Employees
WHERE LastName = 'Patil';
```

14. Show employees hired between 2018 and 2020

```
SELECT FirstName, LastName, HireDate
FROM Employees
WHERE HireDate BETWEEN '2018-01-01' AND '2020-12-31';
```

15. Find employees whose first name starts with 'P'

```
SELECT FirstName, LastName
FROM Employees
WHERE FirstName LIKE 'P%';
```

16. Count employees in each department

```
SELECT d.DepartmentName, COUNT(e.EmployeeID) AS EmployeeCount
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
GROUP BY d.DepartmentName;
```

17. Show employees ordered by hire date (oldest first)

```
SELECT FirstName, LastName, HireDate
FROM Employees
ORDER BY HireDate ASC;
```

18. Show top 5 highest-paid employees

```
SELECT FirstName, LastName, Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 5;
```

19. Find employees whose salary is not equal to 60,000

```
SELECT FirstName, LastName, Salary
FROM Employees
WHERE Salary != 60000;
```

20. Show distinct cities where departments are located

```
SELECT DISTINCT Location
FROM Departments;
```

Intermediate Level

Building proficiency with subqueries, aggregations, joins, and window functions

21. Calculate average salary by department

```
SELECT d.DepartmentName, AVG(e.Salary) AS AvgSalary
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
GROUP BY d.DepartmentName;
```

22. Find employees who worked more than 1000 hours

```
SELECT e.FirstName, e.LastName, ep.HoursWorked
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
WHERE ep.HoursWorked > 1000;
```

23. Show employees with names starting with 'A'

```
SELECT FirstName, LastName FROM Employees WHERE FirstName LIKE 'A%';
```

24. Find employees not in HR department

```
SELECT FirstName, LastName FROM Employees
WHERE DepartmentID NOT IN (102);
```

25. Show employees with salary between 60,000 and 80,000

```
SELECT FirstName, LastName, Salary FROM Employees WHERE Salary BETWEEN 60000 AND 80000;
```

26. Count employees per project

```
SELECT p.ProjectName, COUNT(ep.EmployeeID) AS EmployeeCount
FROM Projects p
JOIN EmployeeProjects ep ON p.ProjectID = ep.ProjectID
GROUP BY p.ProjectName;
```

27. Show employees who don't work on any project

```
SELECT e.FirstName, e.LastName
FROM Employees e
LEFT JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
WHERE ep.ProjectID IS NULL;
```

28. Find department with maximum employees

```
SELECT d.DepartmentName, COUNT(e.EmployeeID) AS EmpCount
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
GROUP BY d.DepartmentName
ORDER BY EmpCount DESC LIMIT 1;
```

29. Show employees who don't work on any project

```
SELECT e.FirstName, e.LastName
FROM Employees e
LEFT JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
WHERE ep.ProjectID IS NULL;
```

30. Find employees who worked on more than one project

```
SELECT e.FirstName, e.LastName, COUNT(ep.ProjectID) AS ProjectCount
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName
HAVING COUNT(ep.ProjectID) > 1;
```

31. Show employees with salary greater than their department average

```
SELECT e.FirstName, e.LastName, e.Salary, d.DepartmentName
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE e.Salary > (
    SELECT AVG(Salary)
    FROM Employees
    WHERE DepartmentID = e.DepartmentID
);
```

32. List employees who joined before their department's oldest project started

```
SELECT e.FirstName, e.LastName, e.HireDate, d.DepartmentName
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE e.HireDate < (
    SELECT MIN(StartDate)
    FROM Projects
);
```

33. Show employees with their project names

```
SELECT e.FirstName, e.LastName, p.ProjectName
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
JOIN Projects p ON ep.ProjectID = p.ProjectID;
```

34. Find employees who worked more than 1000 hours in total

```
SELECT e.FirstName, e.LastName, SUM(ep.HoursWorked) AS TotalHours
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
```

```
GROUP BY e.EmployeeID, e.FirstName, e.LastName
HAVING SUM(ep.HoursWorked) > 1000;
```

35. Show employees with their department and project count

```
SELECT e.FirstName, e.LastName, d.DepartmentName, COUNT(ep.ProjectID) AS
ProjectCount
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
LEFT JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName, d.DepartmentName;
```

36. Find projects with no employees assigned

```
SELECT p.ProjectName
FROM Projects p
WHERE NOT EXISTS (
    SELECT 1 FROM EmployeeProjects ep WHERE ep.ProjectID = p.ProjectID
);
```

37. Show employees with their role in each project

```
SELECT e.FirstName, e.LastName, p.ProjectName, ep.Role
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
JOIN Projects p ON ep.ProjectID = p.ProjectID;
```

38. Find employees who earn the maximum salary in their department

```
SELECT e.FirstName, e.LastName, e.Salary, d.DepartmentName
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE e.Salary = (
    SELECT MAX(Salary)
    FROM Employees
    WHERE DepartmentID = e.DepartmentID
);
```

39. Show employees who worked on projects with budget > 300,000


```
SELECT DISTINCT e.FirstName, e.LastName
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
JOIN Projects p ON ep.ProjectID = p.ProjectID
WHERE p.Budget > 300000;
```

40. Find employees who joined in the same year as Ashish Zope

```
SELECT FirstName, LastName, HireDate
FROM Employees
WHERE EXTRACT(YEAR FROM HireDate) = (
    SELECT EXTRACT(YEAR FROM HireDate) FROM Employees WHERE FirstName = 'Ashish'
    AND LastName = 'Zope'
);
```

41. Show employees with salary difference compared to department average

```
SELECT e.FirstName, e.LastName, e.Salary,
       e.Salary - AVG(e.Salary) OVER (PARTITION BY DepartmentID) AS
DiffFromDeptAvg
FROM Employees e;
```

42. Find employees who worked on the same project together

```
SELECT e1.FirstName AS Emp1, e2.FirstName AS Emp2, p.ProjectName
FROM EmployeeProjects ep1
JOIN EmployeeProjects ep2 ON ep1.ProjectID = ep2.ProjectID AND ep1.EmployeeID <
ep2.EmployeeID
JOIN Employees e1 ON ep1.EmployeeID = e1.EmployeeID
JOIN Employees e2 ON ep2.EmployeeID = e2.EmployeeID
JOIN Projects p ON ep1.ProjectID = p.ProjectID;
```

43. Show employees with their total salary contribution to company

```
SELECT SUM(Salary) AS TotalCompanySalary FROM Employees;
```

44. Find employees who joined in the last 3 years

```
SELECT FirstName, LastName, HireDate
FROM Employees
WHERE HireDate >= CURRENT_DATE - INTERVAL '3 years';
```

45. Show employees with project count greater than department average project count

```
SELECT e.FirstName, e.LastName, COUNT(ep.ProjectID) AS ProjectCount
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName
HAVING COUNT(ep.ProjectID) > (
    SELECT AVG(ProjectCount)
    FROM (
        SELECT COUNT(ProjectID) AS ProjectCount
        FROM EmployeeProjects
        GROUP BY EmployeeID
    ) sub
);
```

46. Show employees with their department and salary rank

```
SELECT e.FirstName, e.LastName, d.DepartmentName,
       RANK() OVER (PARTITION BY d.DepartmentName ORDER BY e.Salary DESC) AS
SalaryRank
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID;
```

47. Find employees who worked on projects starting in 2022

```
SELECT DISTINCT e.FirstName, e.LastName
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
JOIN Projects p ON ep.ProjectID = p.ProjectID
WHERE EXTRACT(YEAR FROM p.StartDate) = 2022;
```

48. Show employees with their department and total hours worked

```
SELECT e.FirstName, e.LastName, d.DepartmentName, SUM(ep.HoursWorked) AS
TotalHours
FROM Employees e
```

```
JOIN Departments d ON e.DepartmentID = d.DepartmentID
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName, d.DepartmentName;
```

49. Find employees who worked on both IT and Finance projects

```
SELECT e.FirstName, e.LastName
FROM Employees e
WHERE EXISTS (
    SELECT 1 FROM EmployeeProjects ep
    JOIN Projects p ON ep.ProjectID = p.ProjectID
    WHERE ep.EmployeeID = e.EmployeeID AND p.ProjectName LIKE '%IT%'
)
AND EXISTS (
    SELECT 1 FROM EmployeeProjects ep
    JOIN Projects p ON ep.ProjectID = p.ProjectID
    WHERE ep.EmployeeID = e.EmployeeID AND p.ProjectName LIKE '%Finance%'
);
```

50. Show employees with their department and cumulative salary rank

```
SELECT e.FirstName, e.LastName, d.DepartmentName, e.Salary,
       SUM(e.Salary) OVER (PARTITION BY d.DepartmentID ORDER BY e.Salary DESC) AS
CumulativeSalary
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID;
```

Advanced Level

Building proficiency with CTEs, window functions, recursive queries, and JSON aggregation

51. Rank employees by salary within department

```
SELECT e.FirstName, e.LastName, d.DepartmentName,
       RANK() OVER (PARTITION BY d.DepartmentName ORDER BY e.Salary DESC) AS
SalaryRank
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID;
```

52. Find top 2 highest-paid employees overall

```
SELECT FirstName, LastName, Salary
FROM Employees ORDER BY Salary DESC LIMIT 2;
```

53. Show cumulative salary by hire date

```
SELECT FirstName, LastName, HireDate,
       SUM(Salary) OVER (ORDER BY HireDate) AS CumulativeSalary
FROM Employees;
```

54. Find employees working on multiple projects

```
SELECT e.FirstName, e.LastName, COUNT(ep.ProjectID) AS ProjectCount
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName
HAVING COUNT(ep.ProjectID) > 1;
```

55. Show department-wise highest-paid employee

```
SELECT DISTINCT ON (d.DepartmentName) d.DepartmentName, e.FirstName, e.LastName,
e.Salary
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
ORDER BY d.DepartmentName, e.Salary DESC;
```

Advanced SQL Interview Questions (PostgreSQL)

56. Use a CTE to find employees earning above department average

```
WITH DeptAvg AS (
    SELECT DepartmentID, AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentID
)
SELECT e.FirstName, e.LastName, e.Salary, d.DepartmentName
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
JOIN DeptAvg da ON e.DepartmentID = da.DepartmentID
WHERE e.Salary > da.AvgSalary;
```

57. Find cumulative project hours per employee using window functions

```
SELECT e.FirstName, e.LastName, ep.ProjectID,
       SUM(ep.HoursWorked) OVER (PARTITION BY e.EmployeeID ORDER BY ep.ProjectID)
AS CumulativeHours
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID;
```

58. Show top 3 highest-paid employees per department

```
SELECT FirstName, LastName, DepartmentID, Salary
FROM (
    SELECT e.*, ROW_NUMBER() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC)
AS rn
    FROM Employees e
) ranked
WHERE rn <= 3;
```

59. Use a recursive CTE to list project timelines month by month

```
WITH RECURSIVE ProjectMonths AS (
    SELECT ProjectID, StartDate AS MonthDate, EndDate
    FROM Projects
    UNION ALL
    SELECT ProjectID, MonthDate + INTERVAL '1 month', EndDate
    FROM ProjectMonths
    WHERE MonthDate + INTERVAL '1 month' <= EndDate
)
SELECT ProjectID, MonthDate FROM ProjectMonths ORDER BY ProjectID, MonthDate;
```

60. Pivot employee count by department

```
SELECT
    COUNT(*) FILTER (WHERE DepartmentID = 101) AS IT_Count,
    COUNT(*) FILTER (WHERE DepartmentID = 102) AS Finance_Count,
    COUNT(*) FILTER (WHERE DepartmentID = 103) AS Risk_Count,
    COUNT(*) FILTER (WHERE DepartmentID = 104) AS Ops_Count,
    COUNT(*) FILTER (WHERE DepartmentID = 105) AS HR_Count
FROM Employees;
```

61. Convert employee details into JSON format

```
SELECT json_build_object(  
  'EmployeeID', EmployeeID,  
  'Name', FirstName || ' ' || LastName,  
  'Department', DepartmentID,  
  'Salary', Salary  
) AS EmployeeJSON  
FROM Employees;
```

62. Aggregate project roles into JSON array per employee

```
SELECT e.FirstName, e.LastName,  
       json_agg(json_build_object('ProjectID', ep.ProjectID, 'Role', ep.Role)) AS  
Projects  
FROM Employees e  
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID  
GROUP BY e.EmployeeID, e.FirstName, e.LastName;
```

63. Find employees who worked on all projects using division logic

```
SELECT e.FirstName, e.LastName  
FROM Employees e  
WHERE NOT EXISTS (  
  SELECT ProjectID FROM Projects  
  EXCEPT  
  SELECT ProjectID FROM EmployeeProjects ep WHERE ep.EmployeeID = e.EmployeeID  
);
```

64. Show department salary distribution percentiles

```
SELECT DepartmentID, FirstName, LastName, Salary,  
       PERCENT_RANK() OVER (PARTITION BY DepartmentID ORDER BY Salary) AS  
SalaryPercentile  
FROM Employees;
```

65. Find overlapping projects by date range

```
SELECT p1.ProjectName AS ProjectA, p2.ProjectName AS ProjectB
FROM Projects p1
JOIN Projects p2 ON p1.ProjectID <> p2.ProjectID
WHERE p1.StartDate <= p2.EndDate AND p2.StartDate <= p1.EndDate;
```

66. Show employees with duplicate last names

```
SELECT LastName, COUNT(*) AS Count
FROM Employees
GROUP BY LastName
HAVING COUNT(*) > 1;
```

67. Find employees with salary above company median

```
WITH SalaryRank AS (
    SELECT Salary, PERCENT_RANK() OVER (ORDER BY Salary) AS pr
    FROM Employees
)
SELECT * FROM Employees WHERE Salary > (
    SELECT Salary FROM SalaryRank WHERE pr = 0.5
);
```

68. Show department with max cumulative salary

```
SELECT d.DepartmentName, SUM(e.Salary) AS TotalSalary
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
GROUP BY d.DepartmentName
ORDER BY TotalSalary DESC LIMIT 1;
```

69. Find employees who never worked with Ashish Zope

```
SELECT e.FirstName, e.LastName
FROM Employees e
WHERE e.EmployeeID <> 1
AND NOT EXISTS (
    SELECT 1 FROM EmployeeProjects ep1
    JOIN EmployeeProjects ep2 ON ep1.ProjectID = ep2.ProjectID
    WHERE ep1.EmployeeID = 1 AND ep2.EmployeeID = e.EmployeeID
);
```

70. Show average hours worked per project role

```
SELECT Role, AVG(HoursWorked) AS AvgHours
FROM EmployeeProjects
GROUP BY Role;
```

71. Use CTE to find employees with more than 1 project

```
WITH MultiProject AS (
    SELECT EmployeeID, COUNT(ProjectID) AS ProjectCount
    FROM EmployeeProjects
    GROUP BY EmployeeID
)
SELECT e.FirstName, e.LastName, mp.ProjectCount
FROM Employees e
JOIN MultiProject mp ON e.EmployeeID = mp.EmployeeID
WHERE mp.ProjectCount > 1;
```

72. Show employees with salary rank across company

```
SELECT FirstName, LastName, Salary,
       DENSE_RANK() OVER (ORDER BY Salary DESC) AS SalaryRank
FROM Employees;
```

73. Find longest-serving employee per department

```
SELECT d.DepartmentName, e.FirstName, e.LastName, e.HireDate
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE e.HireDate = (
    SELECT MIN(HireDate) FROM Employees WHERE DepartmentID = d.DepartmentID
);
```

74. Show employees with salary difference from department average

```
SELECT e.FirstName, e.LastName, e.Salary,
       e.Salary - AVG(e.Salary) OVER (PARTITION BY DepartmentID) AS
```



```
DiffFromDeptAvg
FROM Employees e;
```

75. Find employees who worked on projects with budget > 400,000

```
SELECT DISTINCT e.FirstName, e.LastName
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
JOIN Projects p ON ep.ProjectID = p.ProjectID
WHERE p.Budget > 400000;
```

76. Show JSON array of employees per department

```
SELECT d.DepartmentName,
       json_agg(json_build_object('Name', e.FirstName || ' ' || e.LastName,
                                   'Salary', e.Salary)) AS Employees
FROM Departments d
JOIN Employees e ON d.DepartmentID = e.DepartmentID
GROUP BY d.DepartmentName;
```

77. Use recursive CTE to generate employee hierarchy (manager → employee)

```
WITH RECURSIVE EmpHierarchy AS (
  SELECT EmployeeID, FirstName, LastName, ManagerID, 1 AS Level
  FROM Employees WHERE ManagerID IS NULL
  UNION ALL
  SELECT e.EmployeeID, e.FirstName, e.LastName, e.ManagerID, eh.Level + 1
  FROM Employees e
  JOIN EmpHierarchy eh ON e.ManagerID = eh.EmployeeID
)
SELECT * FROM EmpHierarchy ORDER BY Level;
```

78. Show employees with project count and total hours

```
SELECT e.FirstName, e.LastName,
       COUNT(ep.ProjectID) AS ProjectCount,
       SUM(ep.HoursWorked) AS TotalHours
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName;
```

79. Find departments where all employees earn above 60,000

```
SELECT d.DepartmentName
FROM Departments d
WHERE NOT EXISTS (
    SELECT 1
    FROM Employees e
    WHERE e.DepartmentID = d.DepartmentID
        AND e.Salary <= 60000
);
```

Expert Level (81–95)

81. Find employees who earn above their department average

```
SELECT e.FirstName, e.LastName, e.Salary, d.DepartmentName
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE e.Salary > (
    SELECT AVG(Salary)
    FROM Employees
    WHERE DepartmentID = e.DepartmentID
);
```

82. Show longest-running project by duration

```
SELECT ProjectName, (EndDate - StartDate) AS Duration
FROM Projects
ORDER BY Duration DESC
LIMIT 1;
```

83. Find employees who share the same last name

```
SELECT e1.FirstName, e1.LastName, e2.FirstName AS Colleague
FROM Employees e1
JOIN Employees e2
    ON e1.LastName = e2.LastName AND e1.EmployeeID <> e2.EmployeeID;
```

84. Show department with highest average salary

```

SELECT d.DepartmentName, AVG(e.Salary) AS AvgSalary
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
GROUP BY d.DepartmentName
ORDER BY AvgSalary DESC
LIMIT 1;

```

85. Use recursive CTE to generate a calendar of project days

```

WITH RECURSIVE ProjectDays AS (
  SELECT ProjectID, StartDate AS DayDate, EndDate
  FROM Projects
  UNION ALL
  SELECT ProjectID, DayDate + INTERVAL '1 day', EndDate
  FROM ProjectDays
  WHERE DayDate + INTERVAL '1 day' <= EndDate
)
SELECT ProjectID, DayDate
FROM ProjectDays
ORDER BY ProjectID, DayDate;

```

86. Find employees who worked on the maximum number of projects

```

SELECT e.FirstName, e.LastName
FROM Employees e
WHERE e.EmployeeID IN (
  SELECT EmployeeID
  FROM EmployeeProjects
  GROUP BY EmployeeID
  HAVING COUNT(ProjectID) = (
    SELECT MAX(ProjectCount)
    FROM (
      SELECT EmployeeID, COUNT(ProjectID) AS ProjectCount
      FROM EmployeeProjects
      GROUP BY EmployeeID
    ) sub
  )
);

```

87. Show salary distribution quartiles per department

```

SELECT DepartmentID, FirstName, LastName, Salary,
  NTILE(4) OVER (PARTITION BY DepartmentID ORDER BY Salary) AS Quartile

```

```
FROM Employees;
```

88. Find employees who worked on overlapping projects

```
SELECT e.FirstName, e.LastName, p1.ProjectName, p2.ProjectName
FROM EmployeeProjects ep1
JOIN EmployeeProjects ep2 ON ep1.EmployeeID = ep2.EmployeeID AND ep1.ProjectID <
ep2.ProjectID
JOIN Projects p1 ON ep1.ProjectID = p1.ProjectID
JOIN Projects p2 ON ep2.ProjectID = p2.ProjectID
WHERE p1.StartDate <= p2.EndDate AND p2.StartDate <= p1.EndDate;
```

89. Show employees with salary difference compared to company average

```
SELECT FirstName, LastName, Salary,
       Salary - AVG(Salary) OVER () AS DiffFromCompanyAvg
FROM Employees;
```

90. Find departments where all employees earn above 60,000

```
SELECT d.DepartmentName
FROM Departments d
WHERE NOT EXISTS (
    SELECT 1
    FROM Employees e
    WHERE e.DepartmentID = d.DepartmentID
          AND e.Salary <= 60000
);
```

91. Show employees with their project count and average hours worked

```
SELECT e.FirstName, e.LastName,
       COUNT(ep.ProjectID) AS ProjectCount,
       AVG(ep.HoursWorked) AS AvgHours
FROM Employees e
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName;
```

92. Use CTE to find employees who worked on more than 2 projects

```
WITH ProjectCounts AS (  
    SELECT EmployeeID, COUNT(ProjectID) AS ProjectCount  
    FROM EmployeeProjects  
    GROUP BY EmployeeID  
)  
SELECT e.FirstName, e.LastName, pc.ProjectCount  
FROM Employees e  
JOIN ProjectCounts pc ON e.EmployeeID = pc.EmployeeID  
WHERE pc.ProjectCount > 2;
```

93. Show employees with salary rank across company

```
SELECT FirstName, LastName, Salary,  
       RANK() OVER (ORDER BY Salary DESC) AS SalaryRank  
FROM Employees;
```

94. Find employees who worked on projects with budget above average

```
SELECT DISTINCT e.FirstName, e.LastName  
FROM Employees e  
JOIN EmployeeProjects ep ON e.EmployeeID = ep.EmployeeID  
JOIN Projects p ON ep.ProjectID = p.ProjectID  
WHERE p.Budget > (SELECT AVG(Budget) FROM Projects);
```

95. Show department-wise employee salary percentiles

```
SELECT DepartmentID, FirstName, LastName, Salary,  
       PERCENT_RANK() OVER (PARTITION BY DepartmentID ORDER BY Salary) AS  
SalaryPercentile  
FROM Employees;
```

Critical Thinking & Very Difficult

Expert-level challenges combining complex CTEs, window functions, and multi-step logic

96. Find employees who worked on all projects

```
SELECT e.FirstName, e.LastName  
FROM Employees e
```

```

WHERE NOT EXISTS (
    SELECT p.ProjectID FROM Projects p
    EXCEPT
    SELECT ep.ProjectID FROM EmployeeProjects ep WHERE ep.EmployeeID =
e.EmployeeID
);

```

97. Show department where every employee earns above company average

```

SELECT d.DepartmentName
FROM Departments d
WHERE NOT EXISTS (
    SELECT 1 FROM Employees e
    WHERE e.DepartmentID = d.DepartmentID
    AND e.Salary <= (SELECT AVG(Salary) FROM Employees)
);

```

98. Find employees who never worked with Ashish Sunil Zope

```

SELECT e.FirstName, e.LastName
FROM Employees e
WHERE e.EmployeeID <> 1
AND NOT EXISTS (
    SELECT 1 FROM EmployeeProjects ep1
    JOIN EmployeeProjects ep2 ON ep1.ProjectID = ep2.ProjectID
    WHERE ep1.EmployeeID = 1 AND ep2.EmployeeID = e.EmployeeID
);

```

99. Show project with maximum employee hours contribution

```

SELECT p.ProjectName, SUM(ep.HoursWorked) AS TotalHours
FROM Projects p
JOIN EmployeeProjects ep ON p.ProjectID = ep.ProjectID
GROUP BY p.ProjectName
ORDER BY TotalHours DESC
LIMIT 1;

```

100. Identify projects with budget above company average and high contribution

```

SELECT p.ProjectName, p.Budget, SUM(ep.HoursWorked) AS TotalHours
FROM Projects p

```

```
JOIN EmployeeProjects ep ON p.ProjectID = ep.ProjectID
GROUP BY p.ProjectName, p.Budget
HAVING p.Budget > (SELECT AVG(Budget) FROM Projects)
ORDER BY TotalHours DESC;
```

How to use this document

- Load the **CompanyDB** schema and sample data into a local PostgreSQL instance.
- Run the example queries against the sample data to observe results and tweak queries for variations.
- Convert questions to live coding practice: explain approach aloud, write the query, then optimize and test.

Practice tips

- Start with Beginner questions and ensure you can explain each **JOIN**, **GROUP BY**, and window function used.
- For Intermediate/Advanced, practice writing the same result using at least two different approaches (CTE, subquery, JOIN).
- Time-box attempts (e.g., 10–15 minutes per question) to simulate interview pressure.

Schema Overview

We will create a database schema called **CompanyDB** with the following tables:

```
CREATE DATABASE CompanyDB;
```

1. **Departments** - Stores department details.
2. **Employees** - Stores employee information.
3. **Projects** - Stores project details.
4. **EmployeeProjects** - Many-to-Many relationship between employees and projects.

Table Definitions

1. Departments

- **DepartmentID**: Unique identifier for each department (Primary Key).
- **DepartmentName**: Name of the department.
- **Location**: Office location.

```
-----
-- 1. Departments Table
-----

DROP TABLE Departments CASCADE;
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(100) NOT NULL,
    Location VARCHAR(100)
```

```
);
-- Insert Data
INSERT INTO Departments (DepartmentID, DepartmentName, Location) VALUES
(101, 'IT', 'Pune'),
(102, 'Finance', 'Jalgaon'),
(103, 'Risk Management', 'Bhusawal'),
(104, 'Operations', 'Varangaon'),
(105, 'HR', 'Nashik');
```

2. Employees

- **EmployeeID**: Unique identifier for each employee (Primary Key).
- **FirstName**: Employee's first name.
- **LastName**: Employee's last name.
- **DepartmentID**: Foreign Key referencing Departments.
- **HireDate**: Date employee was hired.
- **Salary**: Current salary.

```
-----
-- 2. Employees Table
-----
```

```
DROP TABLE IF EXISTS Employees CASCADE;
CREATE TABLE Employees (
    EmployeeID    INT PRIMARY KEY,
    FirstName     VARCHAR(50) NOT NULL,
    LastName      VARCHAR(50) NOT NULL,
    DepartmentID  INT REFERENCES Departments(DepartmentID),
    HireDate      DATE NOT NULL,
    Salary        DECIMAL(10,2) NOT NULL,
    ManagerID     INT REFERENCES Employees(EmployeeID)
);
-- Insert Data
INSERT INTO Employees (EmployeeID, FirstName, LastName, DepartmentID, HireDate, Salary, ManagerID) VALUES
(1, 'Ashish', 'Zope', 101, '2017-05-12', 90000.00, NULL),
(2, 'Snehal', 'Patil', 102, '2019-07-10', 75000.00, 1),
(3, 'Rohit', 'Chaudhari', 101, '2020-01-20', 60000.00, 1),
(4, 'Priya', 'Mahajan', 103, '2021-11-05', 82000.00, 2),
(5, 'Amit', 'Deshmukh', 104, '2018-02-15', 65000.00, 2),
(6, 'Neha', 'Zope', 105, '2022-03-01', 55000.00, 3),
(7, 'Sagar', 'Patil', 101, '2016-09-25', 98000.00, NULL),
(8, 'Pooja', 'Chaudhari', 102, '2020-12-11', 72000.00, 7),
(9, 'Vikram', 'Mahajan', 103, '2015-04-18', 88000.00, 7),
(10, 'Kiran', 'Deshmukh', 104, '2019-08-22', 64000.00, 9);
```

3. Projects

- **ProjectID**: Unique identifier for each project (Primary Key).
- **ProjectName**: Name of the project.

- **StartDate:** Project start date.
- **EndDate:** Project end date.
- **Budget:** Project budget.

```
-----
-- 3. Projects Table
-----
```

```
DROP TABLE IF EXISTS Projects CASCADE;
CREATE TABLE Projects (
    ProjectID    INT PRIMARY KEY,
    ProjectName  VARCHAR(100) NOT NULL,
    StartDate    DATE NOT NULL,
    EndDate      DATE,
    Budget       DECIMAL(12,2)
);
-- Insert Data
INSERT INTO Projects (ProjectID, ProjectName, StartDate, EndDate, Budget) VALUES
(201, 'Digital Loan Processing System', '2022-01-01', '2022-12-31', 600000.00),
(202, 'Payment Gateway Integration', '2022-03-01', '2022-09-30', 350000.00),
(203, 'Fraud Detection AI', '2022-06-01', '2022-08-31', 400000.00),
(204, 'Mobile Banking App', '2022-02-15', '2022-07-15', 500000.00),
(205, 'Customer Analytics Dashboard', '2022-04-01', '2022-11-30', 450000.00);
```

4. EmployeeProjects

- **EmployeeID:** Foreign Key referencing Employees.
- **ProjectID:** Foreign Key referencing Projects.
- **Role:** Role of employee in the project.
- **HoursWorked:** Total hours worked.
- **Primary Key:** Combination of EmployeeID + ProjectID.

```
-----
-- 4. EmployeeProjects Table (Many-to-Many)
-----
```

```
DROP TABLE IF EXISTS EmployeeProjects CASCADE;

CREATE TABLE EmployeeProjects (
    EmployeeID   INT REFERENCES Employees(EmployeeID),
    ProjectID    INT REFERENCES Projects(ProjectID),
    Role         VARCHAR(50),
    HoursWorked  INT,
    PRIMARY KEY (EmployeeID, ProjectID)
);
-- Insert Data
INSERT INTO EmployeeProjects (EmployeeID, ProjectID, Role, HoursWorked) VALUES
(1, 201, 'Lead Developer', 1600),
(1, 204, 'Solution Architect', 700),
(2, 202, 'Finance Analyst', 900),
```

```
(3, 201, 'Tester', 600),
(3, 203, 'Data Analyst', 400),
(4, 203, 'Risk Auditor', 500),
(5, 205, 'Operations Specialist', 750),
(6, 205, 'HR Coordinator', 300),
(7, 201, 'System Architect', 1500),
(7, 202, 'Integration Specialist', 800),
(8, 202, 'Business Analyst', 950),
(9, 203, 'Risk Manager', 1000),
(9, 205, 'Controller', 850),
(10, 204, 'Project Manager', 700);
```

✦ About the Author

Ashish Zope | Data Engineer & SQL Specialist

This comprehensive SQL interview guide was created to help professionals master SQL concepts through real-world scenarios and practical questions. All content is original and protected.

🔗 Connect & Follow

- **LinkedIn:** [ashish-zope-627628169](#) – Connect for professional insights
- **Instagram:** [@decode.dataengineer](#) – Follow for daily data engineering tips
- **WhatsApp Channel:** [Join Community](#) – Get updates on new content

📋 Usage Rights

✓ **Permitted:** Personal learning, interview preparation, classroom teaching (with attribution)

✗ **Not Permitted:** Republication, commercial use, or content creation without explicit permission and attribution

© 2025 Ashish Zope. All rights reserved.